



Bài 1 Cấu trúc dữ liệu và giải thuật 2

Công nghệ thông tin (Đại học Xây dựng Hà Nội)



Scan to open on Studocu

Chương 1. NHẬP MÔN CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN 2

(Cho các lớp 67it1&2, Kỳ 2 năm học 2023-2024)

Ôn lại một số kiến thức chính ở học phần 1

I.DANH SÁCH - LIST

1.Khái niệm chung về danh sách LIST

Danh sách là một dãy các phần tử, mỗi phần tử gọi là một nút (node).

Khi các nút chỉ lưu trữ các thông tin dữ liệu và được lưu trữ một cách tuần tự thì gọi là danh sách thường hay tuyến tính, kiểu danh sách này được xử lý theo tuần tự, tuyến tính.

Khi các nút lưu trữ các thông tin dữ liệu và địa chỉ của nút kế tiếp thì gọi là danh sách liên kết đơn.

Khi các nút lưu trữ thông tin dữ liệu và 2 địa chỉ liên kết với nút kế tiếp và nút trước nó thì gọi là danh sách liên kết đôi.

Danh sách được tổ chức, cấu trúc theo kiểu dữ liệu có tên kiểu là *list*.

Kiểu dữ liệu **list** dùng cho danh sách là một trong các kiểu dữ liệu mà được hỗ trợ bởi thư viện Standard Template Library (STL) trong C++, khi trong chương trình có dùng biến kiểu list thì phải viết dòng `#include <list>` vào phần đầu chương trình.

Cấu trúc dữ liệu **List** được hỗ trợ bởi STL là một danh sách liên kết đôi, khác với kiểu cấu trúc **Vector** là kiểu mảng động (dynamic array), là mảng được cấp phát bộ nhớ động bằng **Allocator**(đã học ở phần Con trỏ).

Trong thực tế, một danh sách thông thường sẽ dùng cấu trúc dữ liệu Vector, các thao tác xử lý chủ yếu là thêm phần tử vào cuối mảng và truy cập phần tử theo theo ngẫu nhiên (địa chỉ) nên nhanh và dễ. Tuy nhiên, việc thêm phần tử vào cuối mảng sẽ nhanh hơn việc chèn thêm phần tử vào đầu hay giữa mảng do việc trước khi chèn vào vị trí k, phải lùi các phần tử từ vị trí k đến cuối mảng ra sau 1 vị trí.

List (danh sách liên kết đôi) chỉ nên dùng khi phải thực hiện nhiều lần các phép chèn/xóa phần tử vào đầu hay giữa danh sách, ít phải truy nhập ngẫu nhiên các phần tử. Việc chèn/xóa phần tử trong danh sách liên kết đôi chỉ đơn giản là thay đổi thông tin liên kết ở 2 bên trái-phải của nút. Tuy nhiên, vì các nút không có địa chỉ index thực nên việc truy nhập các phần tử nút chậm, phải duyệt danh sách cho đến khi gặp được phần tử cần thiết.

2.Cài đặt danh sách tuyến tính bằng mảng

Cách sử dụng thông thường nhất đối với danh sách là coi nó như một mảng tĩnh –array hay một mảng động – vector, có dùng đến con trỏ.

Trong mục này xét danh sách như một mảng tĩnh, dùng mảng để cài đặt danh sách.

Khai báo danh sách

Cú pháp :

```
#define Max <số phần tử lớn nhất của dãy>;  
< kiểu dữ liệu > <tên danh sách>[Max];
```

Sau khi khai báo số phần tử tối đa của danh sách Max, tiếp đến cần khai báo kiểu dữ liệu của các phần tử trong danh sách, kiểu dữ liệu vị trí từng phần tử, nếu mỗi phần tử có kiểu cấu trúc struct thì phải định nghĩa struct.

Ví dụ áp dụng danh sách là mảng tĩnh

Xét bài toán cộng 2 đa thức thừa :

$$A= \sum a_i x^i = 5 x^{970} + 12x^{400} + 7$$

$$B = \sum b_j x^j = x^{15} + 8x^3 + 1$$

Lập chương trình tính C=A+B

Ta tổ chức dữ liệu như sau :

Mảng 1(term1):

Hệ số	5	12	7
Số mũ	970	400	0

Mảng 2(term2) :

Hệ số	1	8	1
Số mũ	15	3	0

Mảng 3(term3) :

Hệ số						
Số mũ						

Thuật toán tính hệ số của đa thức kết quả C=A+B :

$$c[k] = \begin{cases} a[i] & \text{nếu số mũ của } a[i] > \text{số mũ của } b[j] \\ b[j] & \text{nếu số mũ của } a[i] < \text{số mũ của } b[j] \\ a[i]+b[j] & \text{nếu số mũ của } a[i] = \text{số mũ của } b[j] \end{cases}$$

3. Danh sách liên kết đơn

3.1. Khái niệm

Một danh sách liên kết là một dãy, một tập các nút, mỗi nút có 2 phần là phần *Dữ liệu* và phần *Liên kết* :

Dữ liệu	Liên kết
---------	----------

Để cấu trúc danh sách liên kết, trước hết phải định vị được phần tử đầu tiên trong danh sách. Khi cho vị trí của một nút bất kỳ trong danh sách, ta có thể xác định được nút mà nó liên kết tới.

Một danh sách liên kết gồm một tập các nút, mỗi nút là một cấu trúc trong đó có hai phần : phần các trường dữ liệu và phần trường liên kết- con trỏ móc nối.

Phần dữ liệu của mỗi nút lưu trữ dữ liệu, phần con trỏ liên kết sẽ chỉ đến địa chỉ nút tiếp theo. Khi con trỏ liên kết không chỉ vào đâu cả, ta nói rằng nó là rỗng (NULL).

Ký hiệu *p là con trỏ, phần dữ liệu của nút này là p->Du_Lieu, phần liên kết là p->next. Để tạo một danh sách rỗng ta gán NULL vào đầu danh sách. Để kiểm tra một danh sách rỗng hay không ta chỉ việc xác định đầu danh sách có giá trị NULL hay không.

3.2. Cài đặt danh sách liên kết

Danh sách liên kết đơn được tạo thành từ các nút, mỗi nút gồm hai thành phần là dữ liệu và liên kết. Thành phần dữ liệu có thể là kiểu dữ liệu cơ bản hoặc tự định nghĩa, phần liên kết là con trỏ, trỏ đến nút tiếp theo.

1. Khai báo kiểu cấu trúc nút :

```
struct Node { int du_lieu;  
              Node* next; }
```

2. Khai báo kiểu cấu trúc danh sách liên kết List

```
struct List { Node* head;  
              Node* tail; };
```

3. Hàm khởi tạo danh sách liên kết rỗng ban đầu

```
void KhoiTaoList(List& list)  
{ list.head=NULL;  
  list.tail=NULL; }
```

3.3. Các hàm thao tác với danh sách liên kết đơn

1. Khởi tạo nút nhận dữ liệu :

```
Node* KhoitaoNode(int du_lieu0)
```

```
{Node* node = new Node;  
  node->du_lieu=du_lieu0;  
  node->next = NULL;  
  return node;  
}
```

2. Thêm nút vào danh sách liên kết

a) Thêm nút vào đầu danh sách

```
void ThemvaoDau(List& list, Node* node)
{ if(list.head == NULL)
    { list.head = node;
      list.tail = node;
    }
  else
    { node->next = list.head;
      list.head = node;
    }
}
```

b) Thêm nút vào cuối danh sách

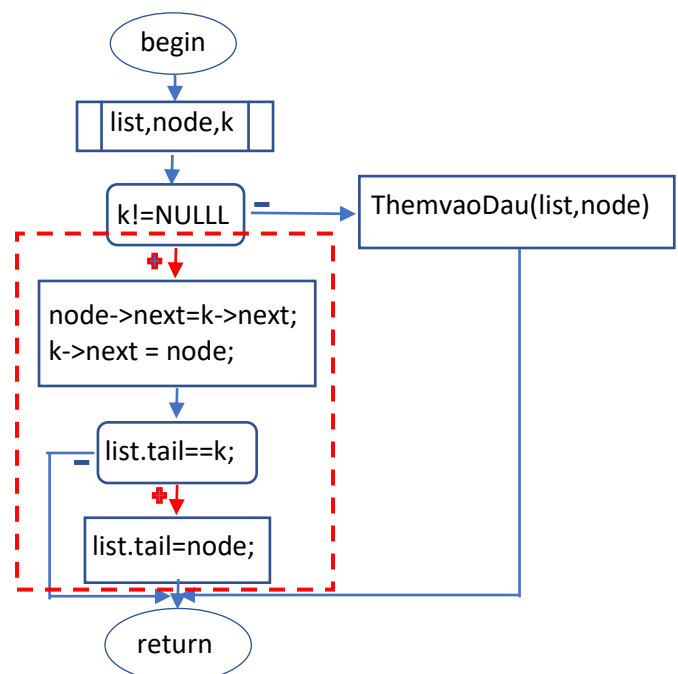
```
void ThemvaoCuoi(List& list, Node* node)
{ if(list.head == NULL)
    { list.head = node;
      list.tail = node;
    }
  else
    { list.tail->next = node ;
      list.tail = node;
    }
}
```

c) Thêm node vào sau nút k (k là tên nút chứ không phải nút ở vị trí k)

```
void ThemNutSauK(List& list, Node* node Node* k)
{ if(k != NULL)
    { node->next = k->next;
      k->next = node;
      if(list.tail == k)
        list.tail = node;
    }
  else
    ThemvaoDau(list, node) ;
}
```

Sơ đồ khối thuật toán cho hàm

ThemNutSauK(List& list, Node* node Node* k) :



3) Xóa nút

a) Xóa nút đầu

```
int XoaNutDau(List& list, int& a)
{ if(list.head != NULL)
    { Node* node = list.head;
      a = node->du_lieu;
      list.head = node->next;
      delete node;
      if(list.head == NULL) list.tail = NULL;
      return 1;
    }
  return 0;
}
```

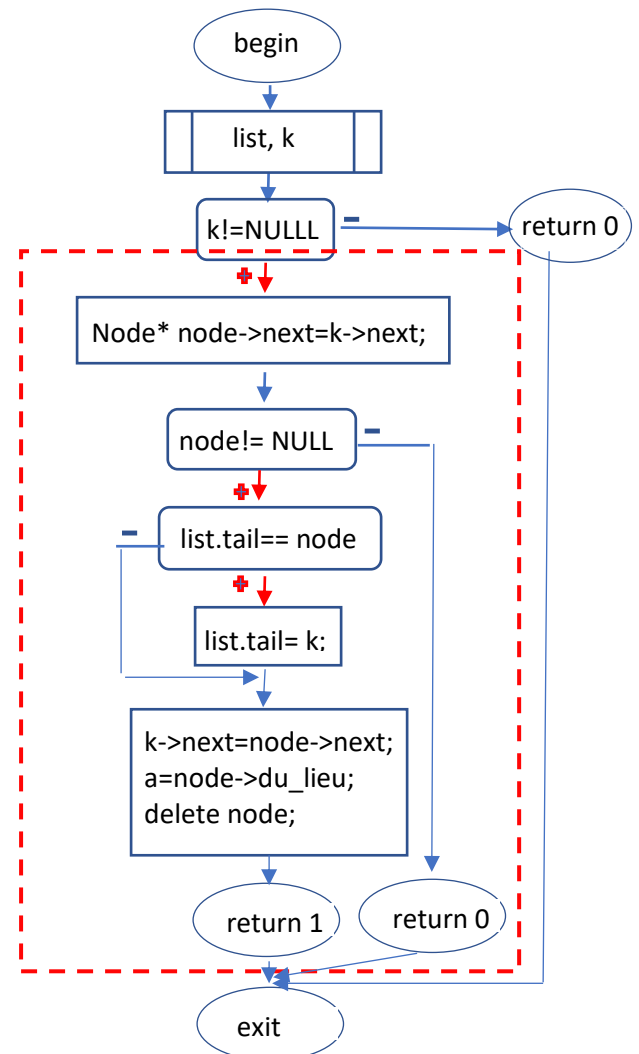
b) Xóa nút sau một nút k xác định

```
int XoaNutSauK(List& list, Node* k, int& a)
{ if(k != NULL)
  { Node* node = k->next; // node là nút sau k
    if (node != NULL)
      { if (list.tail == node) list.tail = k;
        k->next = node->next;
        a = node->du_lieu;
        delete node;
        return 1;
      }
    return 0;
  }
  return 0;
}
```

Sơ đồ khối thuật toán xóa nút sau một nút k xác định

4) Hiện danh sách

```
void HienDS(List list)
{ if (list.head != NULL)
  { Node* node = list.head;
    while (node != NULL)
      { cout<< node->du_lieu<<" ";
        node = node->next;
      }
  }
}
```



5.Lấy giá trị node bất kỳ theo chỉ số nút

```
Node* HienNut(List& list, int chi_so)
{   Node* node = list.head;
    int i=0;
    while (node != NULL && i != chi_so)
        {node = node->next;
          i+ }
    if(i== chi_so && node != NULL)
        return node;
    return NULL; }
```

6)Tìm kiếm phần tử trong danh sách theo giá trị dữ liệu

```
Node* TimNut(List& list, int a)//Tìm theo giá trị dữ liệu
{   Node* node = list.head;
    while (node != NULL && node->data != a)
        node = node->next;
    if(node != NULL) return node;
    return NULL;
}
```

7.Đếm số phần tử của danh sách

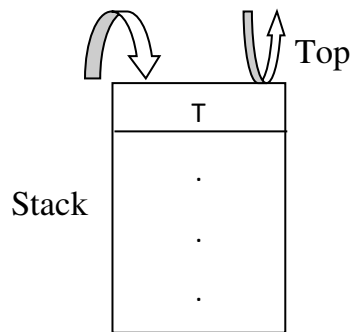
```
int Count(List& list)
{   int n=0;
    Node* node = list.head;
    while (node != NULL )
        {n++;
         node = node->next;}
    return n;
}
```

8.Xóa danh sách

```
void XoaDS(List& list)
{   int a;
    Node* node = list.head;
    while (node != NULL)
        { XoaNutDau(list, a);
          node = list.head;
        }
    list.tail = NULL;
}
```

II. NGĂN XẾP - STACK

Ngăn xếp là một cấu trúc dữ liệu trừu tượng, xử lý theo kiểu vào sau ra trước (last in first out – LIFO) nó là một danh sách hay một dãy bản ghi- cấu trúc, trong đó mỗi phép toán thêm vào hay bớt đi đều thực hiện ở một đầu gọi là đỉnh ngăn xếp.



1. Cài đặt ngăn xếp bằng mảng

dùng mảng 1 chiều **stack** kiểu **int** làm ngăn xếp, một biến **top** lưu chỉ số của phần tử ở đỉnh ngăn xếp, một biến **sizestack** lưu kích thước của ngăn xếp.

```
<kiểu dữ liệu> <tên biến stack>[<int sizestack>];  
int top;
```

2. Các hàm stack cài đặt bằng mảng

2.1. Kiểm tra stack có đầy không (IsFull)

```
bool IsFull(int sizestack)  
{ if(top >= sizestack - 1){ return true; }  
  else  
  { return false; }  
}
```

2.2. Kiểm tra stack rỗng (IsEmpty)

```
bool IsEmpty()  
{ if(top == -1){ return true; }  
  else  
  { return false; }  
}
```

2.3. Thêm phần tử vào đỉnh stack (Push)

```
void Push(int stack[], int gia_tri, int sizestack )  
{  
  if(IsFull(sizestack) == true)  
    { cout<< "\nStack is full – Ngan xep da day ! \n"; }  
  else  
  { ++top; stack[top] = gia_tri; }  
}
```

2.4. Xóa phần tử khỏi đỉnh stack(Pop)

```
void Pop(){
    if(IsEmpty() == true)
        {cout<< "\n Ngan xep trong ! \n";}
    else
        { -- top; }
}
```

2.5. Lấy giá trị phần tử ở đỉnh stack(top)

Để lấy giá trị phần tử ở đỉnh ngăn xếp, ta có thao tác:

```
int Top(int stack[])
{return stack[top];}
```

2.6. Tính số lượng phần tử stack đang có(Size)

Biến **top** lưu chỉ số lớn nhất của ngăn xếp, việc tính số lượng phần tử stack đang có của stack là dùng hàm size() của C++ :

```
int Size()
{ return top + 1; }
```

3.Thư viện chuẩn ngăn xếp

Để sử dụng các thao tác cơ bản trên ngăn xếp nh- những hàm mẫu có sẵn trong thư viện hàm mẫu STL của C++, ta phải khai báo th- viện stack ở phần đầu chương trình :

```
#include <stack>
```

Sau đó khai báo biến: **stack<kiểu dữ liệu> <tên biến stack>;**

Ví dụ : `stack<int> s;`

Các thủ tục trong thư viện chuẩn stack :

1) Đưa(đẩy) phần tử x vào stack : `s.push(x);`

2) Lấy phần tử đỉnh của stack : `int u = s.top();`

Thủ tục này chỉ lấy phần tử đỉnh ra thôi chứ không xóa.

3) Xóa phần tử đỉnh: `s.pop();` Thủ tục này chỉ xóa phần tử đỉnh, không trả về giá trị.

4) Kiểm tra rỗng: trả về true hoặc false : `s.empty();`

4.Lớp ngăn xếp – class stack

Trong định nghĩa lớp stack có các thành viên thuộc tính và phương thức.Các hàm phương thức cơ bản là push() - đẩy phần tử vào đỉnh ngăn xếp và pop() lấy phần tử ở đỉnh ngăn xếp, 2 hàm phương thức này có thể viết bên trong hay bên ngoài định nghĩa lớp. Để khởi tạo một đối tượng mới cho lớp Stack cần dùng thêm Hàm tạo Constructor.

```
#include <iostream>
using namespace std;
```

```
class Stack
```

```

{ private:
    int s[20];
    int k;
public:
    Stack()// constructor
    {    k = 0;  }
    void push(int x);
    int pop()
    {    return s[- k];  }
};// hết định nghĩa class Stack

void Stack::push(int x)
{    s[k++] = x;
    cout<<" k= "<<k<<" \n";
    system("pause");
}

int main()
{    cout << "\n\n    CLASS STACK \n" << endl;
    int i;
    Stack s1,s2,s3;
    cout << " Nhập gia tri i = " ;cin>>i;
    s1.push(i);
    cout << " \n Nhập lại gia tri i = " ;cin>>i;
    s2.push(s1.pop() + i);
    cout << "\n Nhập lại gia tri i = " ;cin>>i;
    s3.push(s2.pop() + i);
    cout << "\n Định của ngăn xếp s3 là "<<s3.pop()<< endl;
    cout << "\n" ;
    system("pause");
    return 0;
}

```

5. Ngăn xếp liên kết đơn

5.1. Khai báo cấu trúc Node

```

struct Node
{ int  du_lieu; Node* next; };

```

5.2. Khai báo cấu trúc ngăn xếp liên kết

```

struct LinkedStack {Node* head; }

```

5.3. Khởi tạo ngăn xếp liên kết rỗng ban đầu

```

void KhoiTaoStack(LinkedStack& s)
{ s.head=NULL;}

```

5.4. Khởi tạo nút mới để nhận dữ liệu cho ngăn xếp liên kết

```
Node* KhoitaoNode(int du_lieu0)
{ Node* p = new Node; //tạo nút p mới và cấp phát bộ nhớ động cho p
  p->du_lieu = du_lieu0; //khởi gán giá trị dữ liệu ban đầu du_lieu cho p
  p->next = NULL; //nút mới tạo chưa trở vào đâu cả nên next=NULL
  return p; // trả về địa chỉ của nút mới được cấp phát
}
```

5.5. Hàm isEmpty - kiểm tra ngăn xếp liên kết có rỗng không

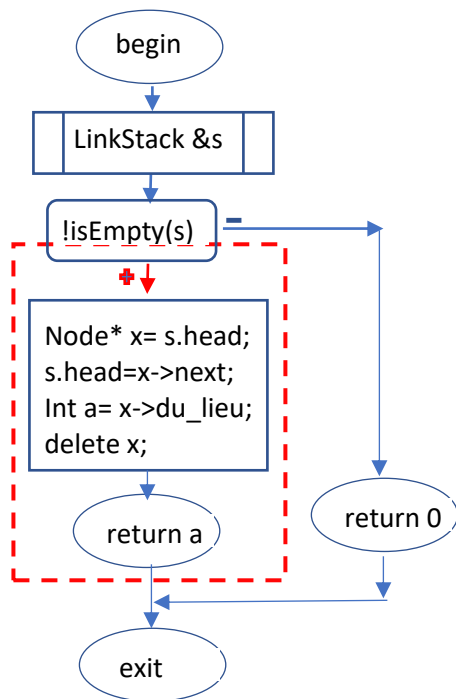
```
bool isEmpty(LinkedStack s)
{ return (s.head == NULL); }
```

5.6. Thêm phần tử vào đỉnh ngăn xếp liên kết

```
void ThemvaoDinh(Linkedstack &s, int du_lieu0)
{ Node* p = KhoitaoNode(int du_lieu0); //tạo một Node mới
  if (isEmpty(s)) // kiểm tra nếu stack rỗng
  { s.head = p; } // nếu rỗng thì đưa nút p vào đỉnh ngăn xếp liên kết
  else //nếu không rỗng,
  { p->next = s.head; // liên kết của nút mới sẽ trở đến đỉnh ngăn xếp l.kết,
    s.head = p; //nút đỉnh ngăn xếp sẽ là nút mới.
  }
}
```

5.7. Xóa phần tử ở đỉnh ngăn xếp liên kết

```
int XoaDinh(LinkedStack &s)
{ if (!isEmpty(s))
  { Node *x = s.head;
    int a = x->du_lieu;
    s.head = x->next;
    delete x;
    cout<<"Xóa thành công !!";
    return a;
  }
  else
  { cout << "Stack rỗng!" << endl;
    return 0; }
}
```



5.8. Hiện dữ liệu của phần tử đỉnh ngăn xếp liên kết

```

int Dinh(LinkedStack s)
{ if (!isEmpty(s))
    { return s.head->du_lieu; }
  else
    { cout << "Stack rỗng!" << endl;
      return 0; }
}

```

5.9 Hiện cả ngăn xếp liên kết

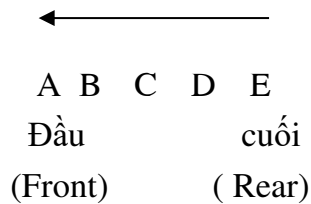
```

void HienStack(LinkedList s)
{ if (s.head != NULL)
  { Node* node = s.head;
    while (node != NULL)
      { cout << node->du_lieu << " ";
        node = node->next; }
  }
}

```

III.HÀNG ĐỢI - QUEUE

Hàng đợi là một danh sách dữ liệu, trong đó các phép thêm phần tử được thực hiện ở cuối danh sách (Rear), còn các phép xử lý như hiện dữ liệu, xoá phần tử được thực hiện ở đầu danh sách (điểm Front)



3.1.Cài đặt hàng đợi bằng mảng

Đặt Q[] – mảng hàng đợi, ngoài mảng Q[n] cần dùng thêm 2 biến **front**(đầu hàng) và **rear**(cuối hàng).

front luôn chỉ vào phần tử đầu của hàng đợi mà có thể lấy ra được, còn **rear** luôn chỉ vị trí cuối hàng đợi, nơi có thể thêm vào phần tử.

```
<kiểu dữ liệu> <tên mảng hàng đợi>[];  
int front, rear;  
int QSize ;// kích thước tối đa của hàng đợi
```

3.2.Các hàm đối với hàng đợi cài đặt bằng mảng

1)Thủ tục Khởi tạo hàng đợi

```
void KhoitaoQ(int Q[], int front, int rear)  
{ front = 0 ; rear = 0 };
```

2)Kiểm tra hàng đợi có rỗng không

```
bool IsEmpty(int front, int rear)  
{ return (front == rear); }
```

3)Cho phần tử ở đầu hàng đợi

```
int FRONT(int Q[], int front)  
{ return Q[front]; }
```

4)Thủ tục thêm phần tử vào cuối hàng đợi:

```
void ThemQ(int Q[], int item, int &rear, int QSize)  
{ if(rear == QSize) {cout<< " Hang doi day \n";}  
  else  
  { Q[rear] = item;  
    rear++; }  
}
```

5)Xoá phần tử ra khỏi đầu hàng đợi :

```
void XoaQ(int Q[], int &front, int rear)  
{ if(front == rear){ cout<< " Hang doi trong rong \n" ;}  
  else  
  { Q[front] = 0; // X óa phần tử ở front  
    front++; }  
}
```

3.3.Thư viện chuẩn hàng đợi

a) Khai báo `#include <queue>`

Viết dòng này ngay sau dòng `#include <iostream>` trong phần đầu chương trình.

b) Khai báo biến hàng đợi q

`queue <kiểu dữ liệu> <tên biến> ;`

Ví dụ : `queue<int> q;`

Trong phần chương trình, sử dụng các hàm của thư viện chuẩn queue như sau.

- 1) Thử tục đưa 1 phần tử x vào queue : `q.push(x);`
- 2) Lấy 1 phần tử đầu của queue (không xóa phần tử này) : `x = q.front();`
- 3) Xóa phần tử đầu (không trả về giá trị): `q.pop();`
- 4) Kiểm tra rỗng: trả về true hoặc false : `q.empty();`

3.4.Phối hợp dùng các hàm trong thư viện chuẩn stack và queue

Chương trình sử dụng thư viện chuẩn stack và queue trong C++, nhập dãy số rồi đưa ra dãy số đã đảo ngược :

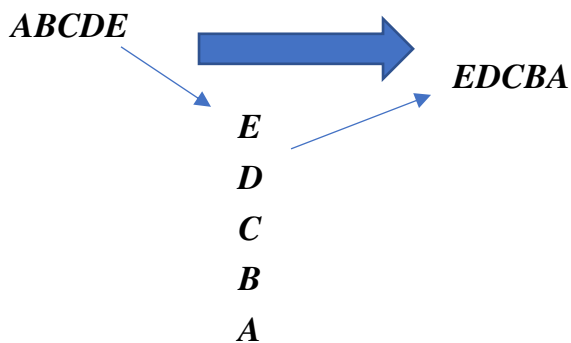
(Chương trình sq1 trong thư mục E:\namhoc19_20\sq1)

```
#include <iostream>
#include <stack>
#include <queue>
#include <iomanip> //Điều khiển cout<<
using namespace std;
int main()
{
    queue<int> q; stack<int> s;
    int x;
    cout<< " \n    Nhập vào các số nguyên : \n ";
    char nhaptiep='Y';
    while (toupper(nhaptiep)=='Y')
    {
        cout<< " \n    Nhập số : "; cin>>x;
        q.push(x);
        cout<<" \n    Có nhập tiếp không (y/n)?" ; cin>>nhaptiep;
    }
    cout<<" \n    Kết quả chuyển vào stack : \n\n"; cout<<setw(20);
    while (not q.empty())
    {
        x = q.front(); q.pop();
        s.push(x); x=s.top();
        cout<<" ; s.x= "<<x;
    }
}
```

```

    cout<<"\n\n"; cout<<"    Ket qua chuyen vao queue : \n\n";
    cout<<setw(20);
    while (not s.empty())
        {   x=s.top(); s.pop();
            q.push(x); cout<<" "; q.x= "<<x;
        }
    cout<<"\n\n";  system("pause");
    return 0;
}

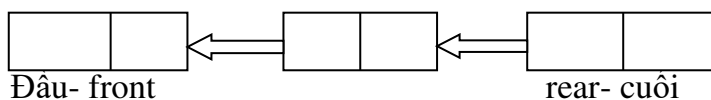
```



3.5.Hàng đợi liên kết đơn

Trong hàng đợi liên kết, các phần tử được nối kết với nhau bằng con trỏ liên kết. Sẽ sử dụng 2 con trỏ **front** và **rear** để trỏ vào đầu và cuối hàng đợi liên kết.

Các thao tác thêm phần tử vào hàng đợi liên kết sẽ thực hiện ở cuối hàng đợi, thao tác xử lý hay xóa phần tử hàng đợi liên kết sẽ thực hiện ở đầu hàng đợi:



1.Cài đặt hàng đợi liên kết

a) Cấu trúc nút-phần tử hàng đợi liên kết

```

struct Node
{   int du_lieu;
    Node *next;
};

```

b)Tạo một cấu trúc của hàng đợi liên kết

```
struct Queue
{
    Node *front, *rear;
};
```

c)Khởi tạo hàng đợi liên rỗng ban đầu

```
void KhoitaoQ(Queue &q)
{
    q.front = q.rear = NULL;
}
```

2.Các hàm với hàng đợi liên kết

a)Tạo nút mới cho hàng đợi liên kết

```
Node *KhoitaoNode( int x )
{
    Node *p = new Node;
    p->next = NULL;
    p->du_lieu = x;
    return p;
}
```

b.Hàm Push –Nhập thêm phần tử vào cuối hàng đợi liên kết

```
void Push(Queue &q, Node *p ){
    if(!q.front) q.front = q.rear = p; //nếu q rỗng thì phần tử thêm vào chính là p.front và p.rear
    else //nếu không thì
    {
        q.rear->next = p; // cho next của q.rear trở tới node p
        q.rear = p; //khi đó p chính là q.rear
    }
}
```

c.Hàm Top- lấy(hiện) phần tử đầu tiên trong hàng đợi Queue liên kết

```
int Top(Queue q )
{
    if(q.front){ return q.front->du_lieu; }
    else return 0; //Nếu hàng đợi rỗng
}
```

d.Hàm Pop- xóa phần tử đầu tiên của hàng đợi liên kết

```
int Pop(Queue &q ) // hàm xóa phần tử đầu trong hàng đợi
{
    if(q.front) //kiểm tra nếu hàng đợi tồn tại thì thực hiện các câu lệnh
    {
        Node *p = q.front; //tạo mới Node p thay thế cho node cần xóa
        q.front = p->next; //trở Next đến phần tử tiếp theo để xóa phần tử đầu
        return p->du_lieu; //gán du_lieu cho p và return nó
        delete p; } // xóa p khỏi hàng đợi
    return 0;// nếu hàng đợi rỗng
}
```

e.Hiện các phần tử trong hàng đợi

```
void Hien(Queue q )
{ Node *p = q.front;
  while(p){ cout<<p->du_lieu <<setw(5);
            p = p->next; }
}
```

f.Xóa hàng đợi

```
void XoaQ(Queue q )
{ Node *p = q.front;
  while(q.front != NULL)
  {   q.front = p->next;
      delete p;
      p = q.front; }
  q.rear = NULL;
}
```

IV. CÁC THUẬT TOÁN SẮP XẾP VÀ TÌM KIẾM

4.1. Sắp xếp thường

Đoạn mã cốt lõi của thuật toán là sắp xếp các phần tử :

```
for (i=1;i<n;i++)  
{ for (j= i+1;j<= n;j++)  
  { if (x[i] <x[j] ){ tg = x[i]; x[i] =x[j]; x[j]:= tg; } }  
}
```

4.2. Sắp xếp ma trận thưa

Ma trận thưa là dạng ma trận có rất ít phần tử có giá trị $\neq 0$. Khi nhập và xử lý chỉ xét các phần tử $\neq 0$.

Ví dụ : cho ma trận thưa A có m hàng, n cột, có q phần tử có giá trị $\neq 0$. Khi đó sẽ tổ chức lưu trữ dữ liệu trong một mảng có 3 cột và (q+1) hàng, ở đây q là số phần tử khác không. Hàng thứ nhất là m,n,q (số hàng, số cột, số phần tử khác 0 của ma trận ban đầu) :

0	m	n	q
1	i	j	$a[i][j] \neq 0$
2			
..	...	...	...
q			

Thuật toán : Bước 1. Hoán vị cột i và cột j

$$b[i][j] = a[j][i] \text{ khi } i \neq j$$

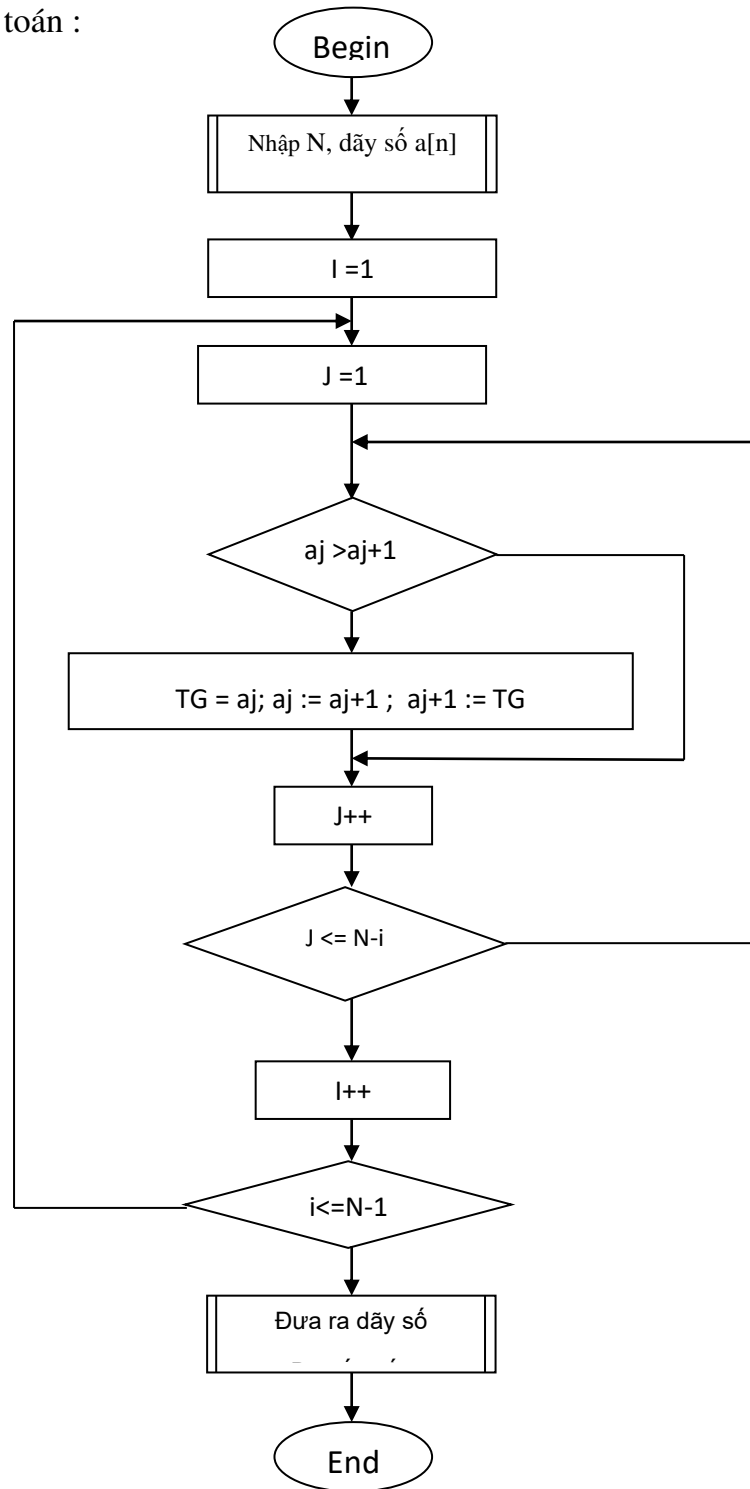
$$b[i][j] = a[j][i] \text{ khi } i=j$$

Bước 2 .Sau đó sắp xếp lại bảng theo cột i tăng dần

4.3. Sắp xếp nổi bọt

Sắp xếp nổi bọt theo trật tự tăng dần được thực hiện theo thuật toán : trong mỗi lần đi qua các dữ liệu nếu không có thứ tự đúng thì sắp xếp ngay hai phần tử liền kề . Theo phương pháp này, mỗi lần duyệt qua một lượt cả danh sách, một số phần tử lớn được nổi lên trên, một số phần tử bé bị chìm xuống dưới. Nếu sắp xếp theo thứ tự giảm dần thì một số phần tử lớn bị chìm xuống dưới, một số phần tử bé được nổi lên trên.

Sơ đồ khối thuật toán :

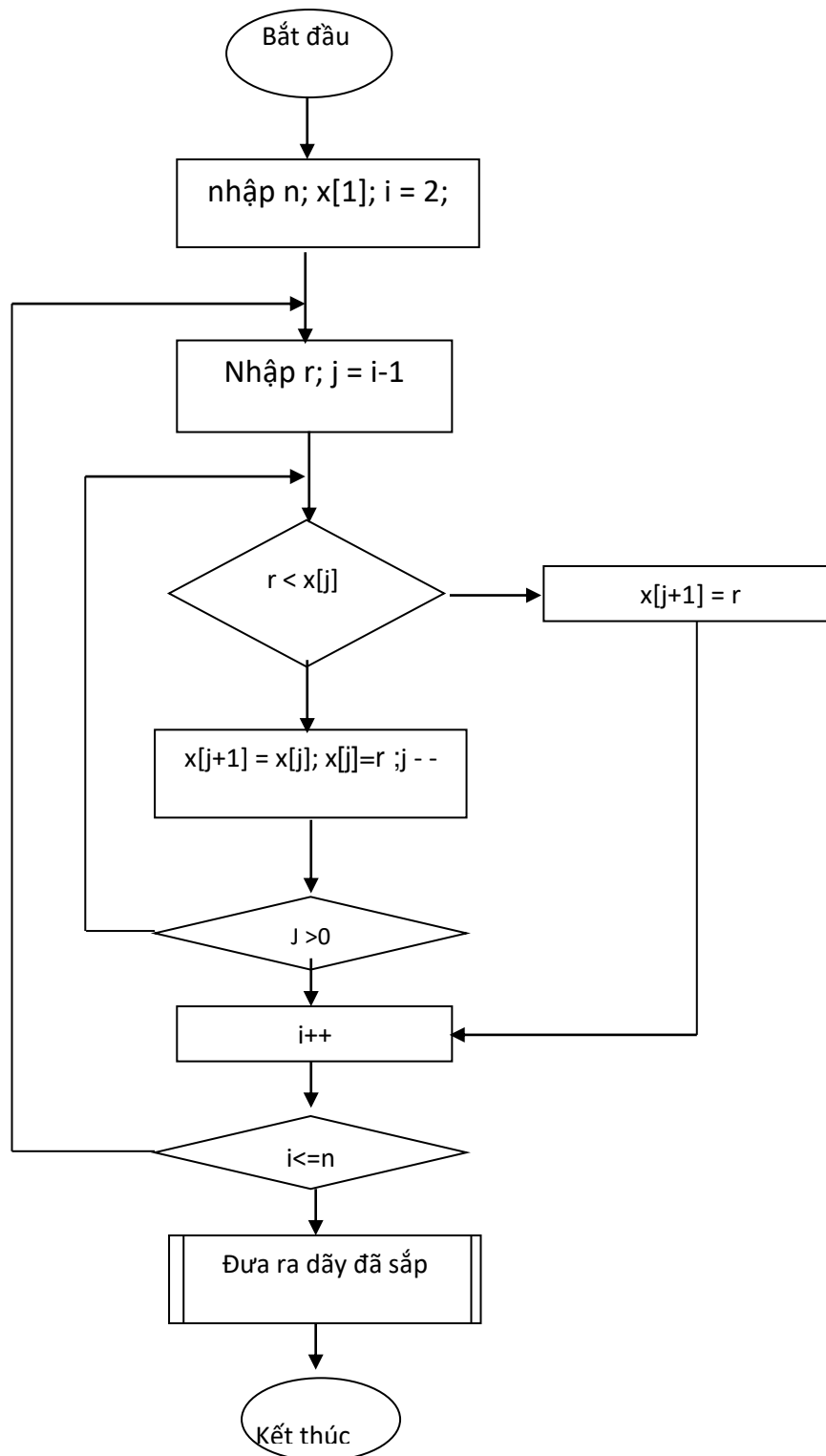


Còn có dạng thuật toán sắp xếp nổi bọt khác, cho j vòng trong chạy từ n-1 cho đến $j \geq i$, so sánh nếu $a[j] > a[j+1]$ thì đổi chỗ $a[j] \leftrightarrow a[j+1]$:

$\text{for}(i=1; i < n; i++) \{ \text{for}(j=n-1; j \geq i; j--) \{ \text{if}(a[j] > a[j+1]) \text{swap}(a[j], a[j+1]); \} \}$

4.4.Sắp xếp chèn – INSERT SORT

Thuật toán này khác biệt ở chỗ không đưa cả dãy số vào trước khi sắp xếp mà đưa vào đến đâu sắp xếp đến đó, giống như khi gọi thí sinh vào lớp xếp vị trí ngồi theo số báo danh.



4.5. Tìm kiếm tuần tự

Tìm kiếm tuyến tính là bắt đầu từ phần tử đầu tiên, tìm một cách tuần tự trong danh sách cho đến khi tìm ra phần tử đã cho hoặc đạt đến cuối danh sách. Kiểu danh sách là mảng các phần tử Kiểu_Ptu.

Ví dụ : Cho danh sách lớp có n người, có các cột mã sinh viên, họ tên, điểm trung bình học kỳ. Vẽ sơ đồ thuật toán và viết chương trình tìm và lập ra danh sách những người có điểm cao nhất lớp, danh sách mới này cũng có các cột mã sinh viên, họ tên, điểm trung bình học kỳ.

4.6. Tìm kiếm trong danh sách liên kết đơn

Tìm kiếm phần tử trong danh sách liên kết đơn cũng là duyệt danh sách từ đầu cho đến khi tìm thấy phần tử hoặc đến hết danh sách (node == NULL) mà không thấy, nếu tìm thấy, sẽ trả về node đó.

Nhập dữ liệu để tìm -> a, sau đó lần lượt tìm các nút, so sánh dữ liệu node.du_lieu với a, nếu bằng(==) thì đưa ra thông báo “đã tìm thấy”, nếu đến hết danh sách không tìm thấy cũng đưa ra tb “không tìm thấy”

```
Node* TimNut(Node* list, int a)
{
    Node* node = list.head;
    while (node != NULL && node->du_lieu != a)
        node = node->next;
    if (node != NULL) {
        cout<<" Da tim thay nut co du lieu ="<<a<<" \n";
        return node; }
    cout<<" Khong tim thay nut co du lieu ="<<a<<" \n";
    return NULL;
}
```

4.7. Tìm kiếm theo chia đôi (nhị phân)

1) Ý tưởng : Tìm kiếm nhị phân đã được nghiên cứu từ lâu trong toán học tính toán.

Muốn tìm một phần tử đã biết chắc chắn có trong một tập Δ , lần thứ nhất ta chia đôi tập Δ thành A_1, A_2 ; Lúc đầu tìm phần tử trong A_1 , nếu tìm thấy thì kết thúc, nếu không thấy thì chia đôi A_2 thành B_1, B_2 ; tiếp tục tìm phần tử trong B_1 , nếu tìm thấy thì kết thúc, nếu không thấy thì chia đôi B_2 thành C_1, C_2 ... quá trình tiếp tục cho đến khi tìm được phần tử.

Theo cách chia đôi : tập A_i có lực lượng = $1/2 \Delta \rightarrow 1/2^1$

B_i có lực lượng = $1/4 \Delta \rightarrow 1/2^2$

C_i có lực lượng = $1/8 \Delta \rightarrow 1/2^3$

Ở bước thứ n, tập được chia đôi có lực lượng $\rightarrow 1/2^n$

Với $n=10$ thì $2^{10}=1024$, nghĩa là ta đã xét được trong tập Δ có hơn một nghìn phần tử.

2) Ứng dụng

Tìm phần tử trong danh sách $x_1, x_2, \dots, x_n$ được sắp xếp theo thứ tự tăng dần. found sẽ có giá trị đúng khi tìm được và mid(middle – điểm giữa) là vị trí điểm chia đôi các phần để tìm.

```

1. found = FALSE; first = 1; last = n ;//first – điểm đầu, last- điểm cuối
2. while (( first <=last )&&(! found ))
    { mid = (first + last)/2;
      if (item < x[mid] ) last = mid-1 ; //item – phần tử cần tìm
      else
        { if(item > x[mid]) first = mid+1;
          else found = TRUE;    }
    }

```

V.ĐỆ QUY

Đệ quy là hiện tượng một thủ tục hay một hàm tham trỏ đến chính nó.

Một thủ tục đệ quy hay hàm đệ quy gồm hai phần:

1. **Phần neo** (anchor – mỏ neo dựng neo thuyền, tàu bè) trong đó tác động của hàm hay thủ tục được đặc tả cho một hay nhiều tham số.
2. **Phần đệ quy**, trong đó **tác động cần được thực hiện cho giá trị hiện thời của các tham số được định nghĩa bằng các tác động hay giá trị được định nghĩa trước đó.**

Ví dụ 1: để tính giai thừa có thể mô tả đệ quy :

ở đây phần **neo** là **$0! = 1$**

đệ quy là với $n > 0$ thì $n! = n * (n-1)!$

Kết quả được tính ngược và cho giá trị cuối cùng

$n! = n * (n-1)!$

$(n-1)! = (n-1) * (n-2)!$

$(n-2)! = (n-2) * (n-3)!$

.....

$3! = 3 * (3-1)!$

$2! = 2 * (2-1)! = 2 * 1!$

$1! = 1 * (1-1)! = 1 * 0!$

$0! = 1$ Cuối cùng đi đến phần neo

Viết hàm đệ quy như sau :

```

int giaiThua(int n)
{ if(n==0)
  { return 1 ;}
  return n*giaiThua(n-1);
}

```

Ví dụ 2 : số fibonacci

1

1

2 với $n > 2$, mỗi số sau là tổng của 2 số trước nó :

3

5

8 Neo $\left\{ \begin{array}{l} \text{Fib}(1)=1 \\ \text{Fib}(2)=1 \end{array} \right.$

Đệ quy : với $n > 2$, $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$

```
int Fib(int n)
{
    if(n<=2)
        {return 1;}
    return (Fib(n-1)+ Fib(n-2));
}
```

Bài tập : dùng hàm đệ quy viết chương trình C++ tính nhị thức Newton :

$$C_{nk}^n = \frac{N!}{K!(n-k)!}$$

hesoNewton = giaithua(N)/(giaithua(K)*giaithua(N-K));

Chứng minh Bài toán Tháp Hà Nội có lời giải

Phải dùng đệ quy (dùng quy nạp):

1) với $n(\text{số đĩa})=1$ thì chuyển được 1 đĩa từ cọc A sang B/C

2) Giả thiết với $n-1$ đĩa đã chuyển được từ A sang C

3) Với n đĩa, sẽ chuyển được $n-1$ đĩa sang C, còn lại 1 đĩa lớn nhất ở cọc A chuyển sang B và chuyển $n-1$ đĩa từ cọc C sang cọc B

VI. ĐỘ PHỨC TẠP THUẬT TOÁN

1. Khái niệm độ phức tạp của thuật toán

Một thuật toán có thể được cài đặt trong máy theo các ngôn ngữ lập trình khác nhau, mỗi cách đòi hỏi phải dùng số lượng bộ nhớ và thời gian tính toán nhất định. Cách nào mà sử dụng ít bộ nhớ nhất và thời gian tính toán ít nhất thì nó là hiệu quả nhất. Ngày nay bộ nhớ không còn là vấn đề nòng cốt ta chỉ quan tâm đến thời gian tính toán. Yếu tố quan trọng nhất ảnh hưởng đến thời gian tính toán là khối lượng thông tin đầu vào.

Ký hiệu T là thời gian thực hiện của thuật toán, khi đó $T(n)$ là một hàm số phụ thuộc vào số lượng đầu vào $n : T(n)$.

Khái niệm **độ phức tạp của thuật toán** được hiểu theo ý nghĩa là độ lớn của $T(n)$, được ký hiệu bằng chữ O (đọc là ô lớn) như sau : giả sử n là số nguyên dương, $T(n)$ và $f(n)$ là các hàm thực không âm, khi đó $T(n)=O(f(n))$ nếu và chỉ nếu tồn tại các hằng dương c và n_0 sao cho $T(n) \leq cf(n)$ với mọi $n \geq n_0$.

Theo định nghĩa trên, hàm $f(n)$ là **cận trên** của $T(n)$.

Một thuật toán có thời gian thực hiện $T(n) = O(f(n))$ - ta nói rằng nó có thời gian thực hiện cấp $f(n)$. Hàm $O(f(n))$ đọc là O -lớn của $f(n)$, tiếng Anh là Big-O

Viết lại định nghĩa : $T(n)=O(f(n))$ nếu và chỉ nếu tồn tại các hằng dương c và n_0 sao cho $T(n) \leq cf(n)$ với mọi $n \geq n_0$

Ví dụ. với bài toán nhập vào từ bàn phím một dãy $a[]$ có n số, đưa ra số lớn nhất (max), thời gian thực hiện $T(n)=3n$, theo định nghĩa ta có thể viết :

$$T(n)=O(n) \text{ với } c=3, n_0>2, T(n) \leq 3n \text{ với mọi } n \geq 2$$

Trở lại phần trên, mỗi thao tác cơ bản sẽ có thời gian thực hiện với độ phức tạp là $O(1)$;

Thuật toán trong ví dụ 1 có thời gian thực hiện với độ phức tạp là $O(n)$.

2. Giải thích ý nghĩa của hàm O lớn (Big O).

Khi nghiên cứu đánh giá độ phức tạp của thuật toán, độ hiệu quả của thuật toán người ta xét độ đo về khía cạnh ước lượng toán học chứ không xét theo góc độ một chương trình dùng một ngôn ngữ cụ thể cài đặt thuật toán đó. Chương trình chỉ dùng để xác minh cho độ phức tạp đã tính toán ra mà thôi.

Một thuật toán có thể được lập trình bằng nhiều ngôn ngữ khác nhau như Pascal hay C++, C#, Java..., có thể chạy trên máy 32 bit hay 64 bit... nên thời gian thực hiện chương trình có thể khác nhau. Vì vậy các nhà khoa học máy tính tiền bối đã đưa ra khái niệm hàm **O - lớn để đánh giá khái quát thời gian thực hiện một thuật toán, có tính chất ước lượng chứ không phải hoàn toàn chính xác, độ phức tạp mang tính tiệm cận (cận trên hay cận dưới).**

Như trong ví dụ thuật toán trên, dù được lập trình bằng Pascal hay C++ hay bằng ngôn ngữ gì nữa thì thời gian thực hiện nó cũng chỉ có thể là $3n$, $4n$ hay không quá cn với c là một hằng nguyên bất kỳ.

Vì vậy, độ phức tạp của thuật toán được định nghĩa là độ lớn của hàm $T(n)$, được gọi bằng hàm O lớn của hàm $f(n) : T(n)=O(f(n))$, nếu và chỉ nếu tồn tại một hằng số $c>0$ và một số $n_0>0$ sao cho $T(n) \leq cf(n)$ với mọi $n \geq n_0$; trong đó $T(n)$ và $f(n)$ là các hàm thực không âm.

Trong lý thuyết độ phức tạp thuật toán, người ta đưa ra một số cấp thời gian chuẩn thực hiện thuật toán như sau :

$O(1)$ -Độ phức tạp hằng số, khi số phép tính/thời gian chạy chương trình/dung lượng bộ nhớ không phụ thuộc đầu vào n , ví dụ : các thao tác hệ thống, hay mỗi phép $+, -, *, /, \%, \&\&, ||, !$

$O(\log(n))$ - độ phức tạp logarit. Ví dụ thuật toán Euclid tìm USCLN, với $n = 8$ thì $\log(8)=3$.

$O(n)$ - độ phức tạp tuyến tính, khi số phép tính/thời gian chạy chương trình/dung lượng bộ nhớ phụ thuộc đầu vào n .

$O(n\log n)$ - độ phức tạp $n\log n$ (với logarit thập phân $\log(n)$ thì $O(n\log n) < O(n)$)

Chú ý là độ phức tạp logarit $O(\log n) < O(n\log n) < O(n)$, xem :

	LOG	n	LOG(n)
logarit nhị phân	$\lg(n)$	8	3
logarit tự liên -e	$\ln(n)$	8	2.079441542
logarit thập phân	$\log(n)$	8	0.903089987

$O(P(n))$ - độ phức tạp đa thức, P là đa thức bậc từ 2 trở lên. Ví dụ bài toán tính định thức ma trận.

$O(n^2)$ - độ phức tạp bình phương

$O(2^n)$ - độ phức tạp số mũ (2^n)

Với các chỉ tiêu trên đây, ta đã có một độ đo để đánh giá, so sánh thuật toán nào tốt hơn thuật toán nào khi giải quyết 1 bài toán..

3. Các quy tắc đánh giá thời gian thực hiện thuật toán

Quy tắc tổng : nếu $T_1(n) = O(f_1(n))$ và $T_2(n) = O(f_2(n))$
thì $T_1(n) + T_2(n) = O(\max(f_1(n), f_2(n)))$

Thật vậy :

vì $T_1(n) = O(f_1(n))$ nên $\exists c_1, n_1$ sao cho $T_1(n) \leq c_1 f_1(n)$ với $\forall n \geq n_1$, ($\exists$ đọc là *tồn tại*)

$T_2(n) = O(f_2(n))$ nên $\exists c_2, n_2$ sao cho $T_2(n) \leq c_2 f_2(n)$ với $\forall n \geq n_2$, ($\forall$ đọc là *với mọi*)

Đặt $n_0 = \max(n_1, n_2)$, khi đó với $\forall n \geq n_0$ ta có :

$$T_1(n) + T_2(n) \leq (c_1 + c_2) \max(f_1(n), f_2(n)) = c \max(f_1(n), f_2(n)) \text{ với } c = c_1 + c_2 \\ = O(\max(f_1(n), f_2(n)))$$

Tổng quát : khi một bài toán được chia thành nhiều bài toán con thì **độ phức tạp** thuật toán của bài toán sẽ lấy theo cấp **độ phức tạp lớn nhất** trong các cấp độ thành phần.

4. Cách tính độ phức tạp của một thuật toán.

Để đánh giá độ phức tạp thực hiện một *thuật toán*/chương trình, ta phải biết được từng phép toán/câu lệnh thực hiện bao nhiêu lần.

Tuỳ theo loại lệnh, ta có các chỉ tiêu đánh giá như sau:

1. Lệnh gán ($=, :=$), lệnh nhập dữ liệu ($\text{cin} >>, \text{read}$) , lệnh hiện dữ liệu ($\text{cout} <<, \text{print}, \text{write}$) có thời gian thực hiện là $O(1)$;

2. Lệnh phức hợp để trong cặp $\{ \dots \}$, $\text{begin} \dots \text{end}$ được đánh giá theo quy tắc tổng ở trên ;

3. Lệnh $\text{if} <\text{điều kiện}> \text{s1 else s2 ;}$

độ phức tạp là $O(\max(f_1(n), f_2(n)))$, trong đó $f_1(n), f_2(n)$ là cận trên của độ phức tạp s_1, s_2 .

4. Lệnh `switch case`, `CASE` được đánh giá như lệnh `if`.

5. Lệnh lặp `for`, `while`, `do...while`, `repeat ...until`, `while..do...` : nếu độ phức tạp thực hiện thân vòng lặp là $f(n)$ và số lần lặp tối đa là $g(n)$ thì độ phức tạp thực hiện một lệnh lặp là $O(f(n)*g(n))$.

6. Đối với hàm đệ quy.

Xét hàm đệ quy tính giai thừa sau.

```
int giaiThua(int n)
{ if (n <= 1)
    return 1;
  return n*giaithua(n-1);
}
```

Ta đặt độ phức tạp thực hiện hàm với n là $T(n)$. Với $n=1$ thì độ phức tạp thực hiện hàm là $O(1)$. Với $n > 1$, độ phức tạp thực hiện hàm chính là thời gian thực hiện lệnh gán `giaithua = n*giaithua(n-1)` và `return giaiThua`, tức là bằng $O(1)$ cộng với $T(n-1)$.

Ta có quan hệ đệ quy :

$T(1) = O(1)$;

$T(n) = O(1) + T(n-1)$

Ta bắt đầu từ $T(n)$ cho đến $T(n-1), T(n-2), \dots, T(2), T(1)$:

$T(n) = O(1) + O(1) + \dots + O(1) + T(1) = (n-1)O(1) + T(1) = nc_1 = O(n)$.

Thuật toán Euclid - tìm ước số chung lớn nhất

```
#include <iostream>
```

```
using namespace std;
```

```
int uscln(int m, int n)
```

```
{ int r ;
```

```
  r = m % n;
```

```
  while ( r != 0){ m = n; n = r; r = m % n ; }
```

```
  return n;
```

```
}
```

```
int main()
```

```
{ cout << "\n\n  THUAT TOAN EUCLID TIM USCLN " << endl;
```

```
  int A,B;
```

```
  cout<< "\n Nhap A,B " ; cin>>A>>B;
```

```
  cout<<" \n USCLN cua "<<A<<" va "<<B<<" la : ";
```

```
  cout << uscln(A, B)<<" \n\n";
```

```
  system("pause");
```

```
  return 0;
```

```
}
```

Xét thời gian thực hiện trong thân thuật toán : **while (r != 0){ m = n; n = r; r = m % n ; }**

$$m = n.q_1 + r_1, \quad 0 \leq r_1 < n$$

$$n = r_1.q_2 + r_2, \quad 0 \leq r_2 < r_1$$

Nếu $r_1 > n/2$ thì suy ra $q_2 = 1$ (vì nếu $q_2 \geq 2$ thì $(số > n/2) * q_2 > n$ là không thể được)
, tức là $n = r_1 + r_2, \Rightarrow r_2 < n/2$

Còn nếu $r_1 < n/2$ thì cũng suy ra $r_2 < n/2$ (vì $r_2 = n \% r_1$, phần dư phải nhỏ hơn số chia).

Nghĩa là trong mọi trường hợp, $r_2 < n/2$.

Ví dụ : $m=20, n=12$;

Lần thực hiện thứ nhất : $m=20; n=12, r_1 = m \% n = 8$; vì $(r_1 \neq 0)$ nên lặp while

Lần thực hiện thứ hai : $m=12; n=8, r_2 = m \% n = 4$; vì $(r_2 \neq 0)$ nên while

....

Như vậy, cứ sau hai lần thực hiện thân vòng lặp thì số d sẽ nhỏ hơn một nửa của n.

Giả sử k là số nguyên lớn nhất sao cho $2^k \leq n$. Từ bất đẳng thức $2^k \leq n$ ta có $k \leq \log_2 n$.

(xem lại cách tính logarit của 1 số : $n=2^k \Rightarrow k = \log_2 n$)

Kết quả , độ phức tạp thuật toán sẽ là $O(\log_2 n)$.

Độ phức tạp của thuật toán sắp xếp theo nổi bọt

Theo sơ đồ, ta chỉ cần đếm vòng lặp theo i và j.

Vòng lặp ngoài chạy theo i từ 1 đến (n-1)

Với mỗi i , vòng lặp trong j sẽ chạy từ 1 đến (n-i) ,

i = 1, vòng lặp j trong chạy n-1 lần

i = 2, vòng lặp j trong chạy n- 2 lần

.....

i = n-1, vòng lặp j trong chạy n-(n-1) = 1 lần

như vậy tổng số lần lặp chính là tổng của n số tự nhiên đầu tiên

$$1+2+3+\dots+(n-2)+(n-1)+n$$

, có công thức là :

$$n(n-1)/2 = n^2/2 - n/2$$

Cộng thêm n lần lặp nhập dữ liệu và n lần hiển dữ liệu cùng với một số thao tác hệ thống, bỏ qua các hằng số, độ phức tạp của thuật toán sắp xếp nổi bọt sẽ là $O(n^2)$.

Với độ phức tạp này cho ta thấy, khi n đủ lớn thì thuật toán sắp xếp nổi bọt là không hiệu quả.

6. Một số ký hiệu độ phức tạp khác

Ngoài ký hiệu O-lớn, người ta còn dùng thêm một số ký hiệu độ phức tạp khác, tùy theo mức độ đánh giá độ phức tạp, dựa theo từng định nghĩa.

6.1 nhỏ- o(.)

Định nghĩa 1: **$T(n)=o(f(n))$** nếu với mọi hằng số c, tồn tại hằng số $n_0 > 0$ sao cho $T(n) < cf(n)$ với mọi $n \geq n_0$.

Như vậy, $o(.)$ là **cận trên** của thời gian thực hiện thuật toán.

Ngoài ra, còn định nghĩa khác cho $o(.)$:

Định nghĩa 2 : $T(n)=o(f(n))$ nếu $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = 0$

6.2. Độ phức tạp Omega - $\Omega(n)$

Định nghĩa : $T(n)=\Omega(f(n))$ nếu tồn tại hằng số $c, n_0 > 0$ sao cho $T(n) \geq cf(n)$ với mọi $n \geq n_0$.

Như vậy, $\Omega(n)$ - độ phức tạp Omega là **cận dưới** của thời gian thực hiện thuật toán.

6.3. Độ phức tạp Teta- $\Theta(.)$

Định nghĩa : $T(n) = \Theta(f(n))$ nếu $T(n) = O(f(n))$ và $T(n) = \Omega(n)$.

Như vậy, độ phức tạp **Teta- $\Theta(.)$** là khi cận trên **$O(.)$** và cận dưới **$\Omega(.)$** trùng với nhau.